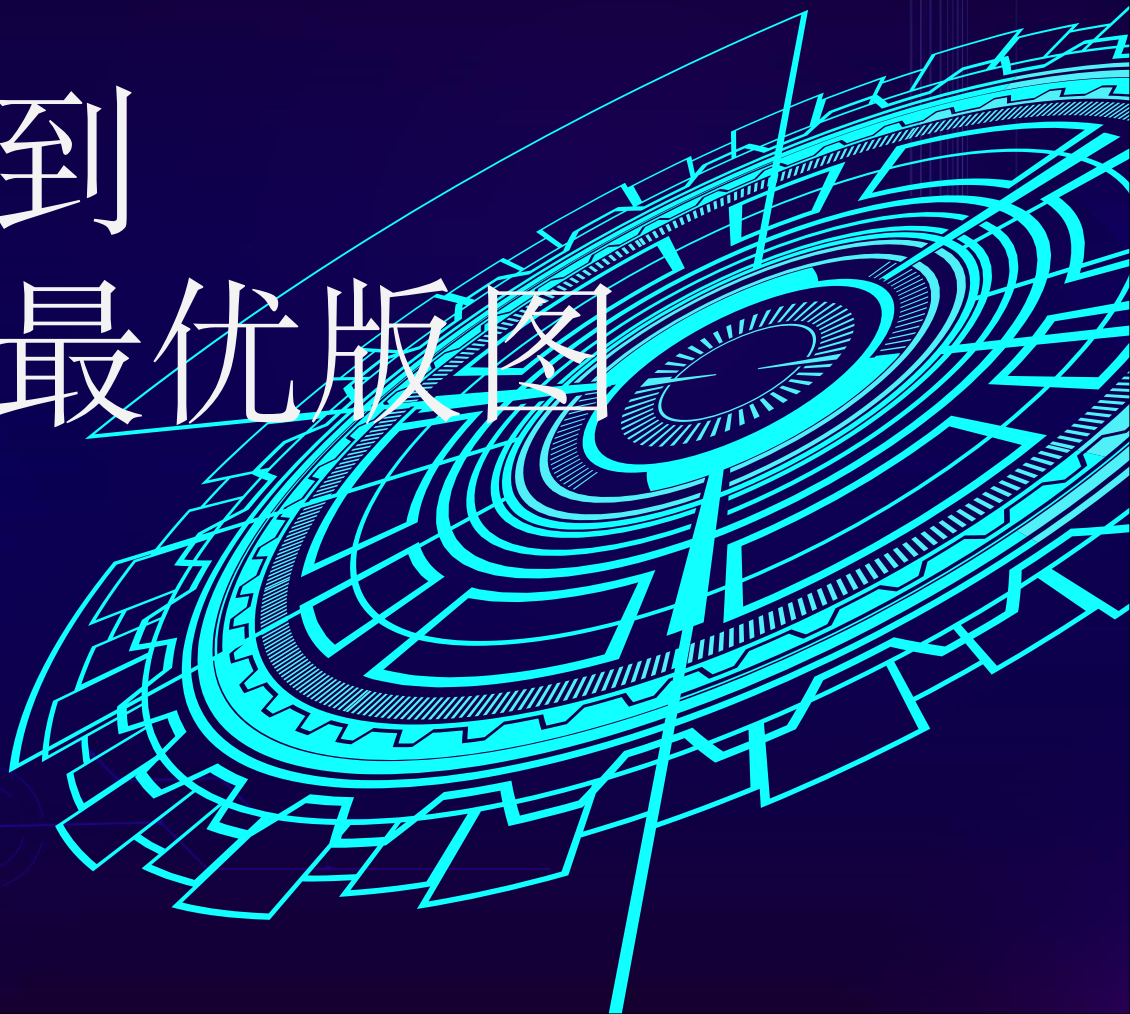


从热运动到 集成电路最优版图

-----基于模拟退火
算法的 VLSI 宏单
元布局优化研究



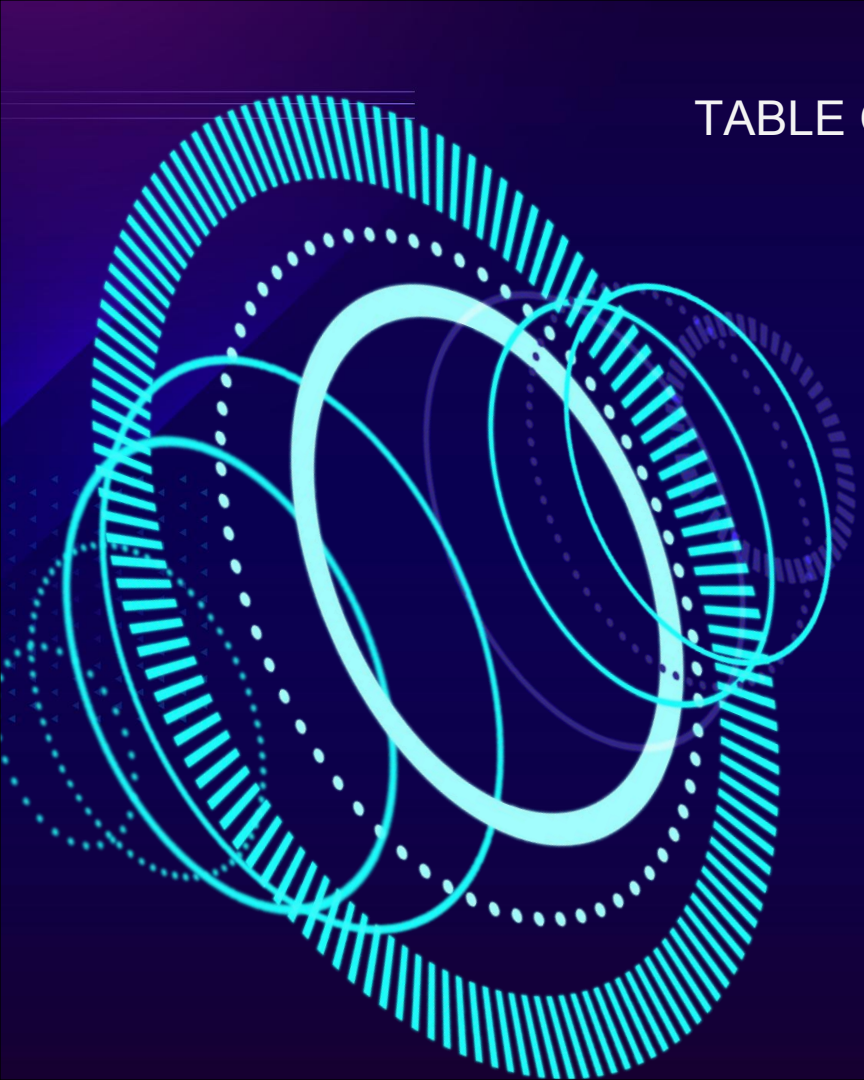


TABLE OF CONTENTS

01 项目背景

02 数学建模

03 核心算法

04 数据可视化与结果分析

05 模型优化与对比

06 现实意义



PART1
项目背景



超大规模集成电路VLSI

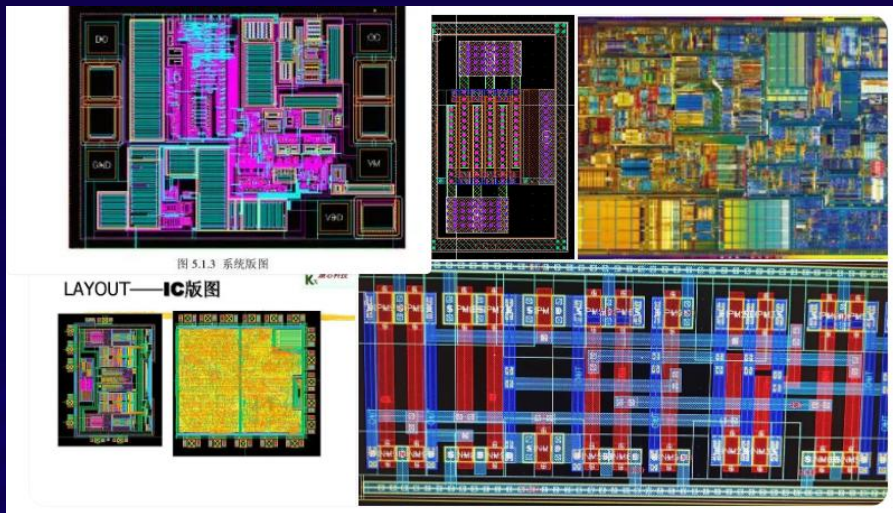
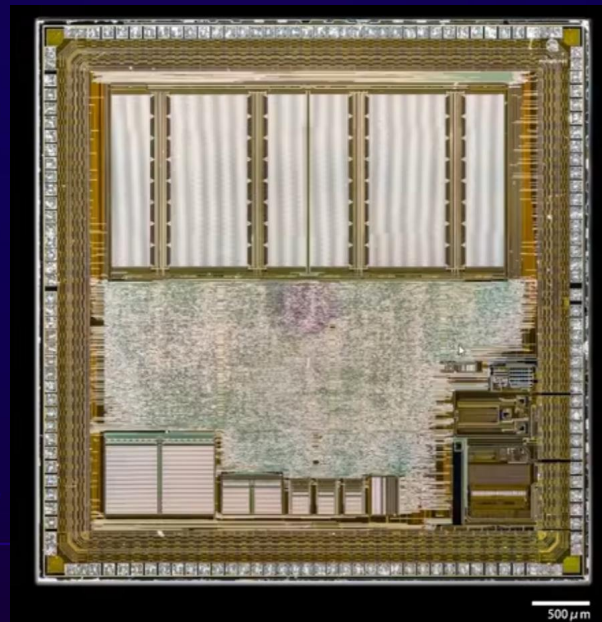
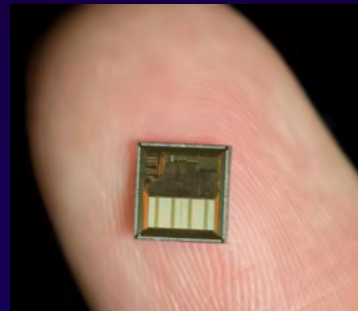


图 5.1.3 系统版图



宏单元 (Macro Blocks)

指体积较大、形状固定的硬核模块 (IP)，如 SRAM 存储阵列、CPU 核心、PLL 等，数量在几十到几百个。

为什么重要？

宏单元就像建筑的“承重墙”，它们的位置一旦定死，整个芯片的PPA（性能、功耗、面积）就基本定型了



核心矛盾： 仅仅“把东西做小”已经不够了， 必须“把东西摆对位置”

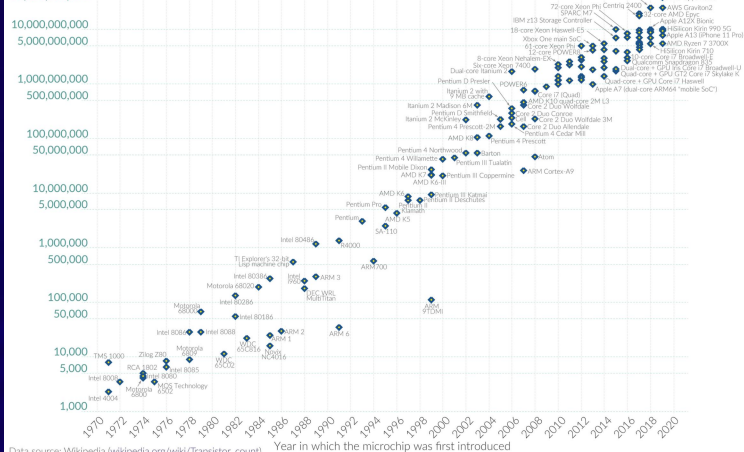
Moore's Law: The number of transistors on microchips doubles every two years

This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count

50,000,000,000

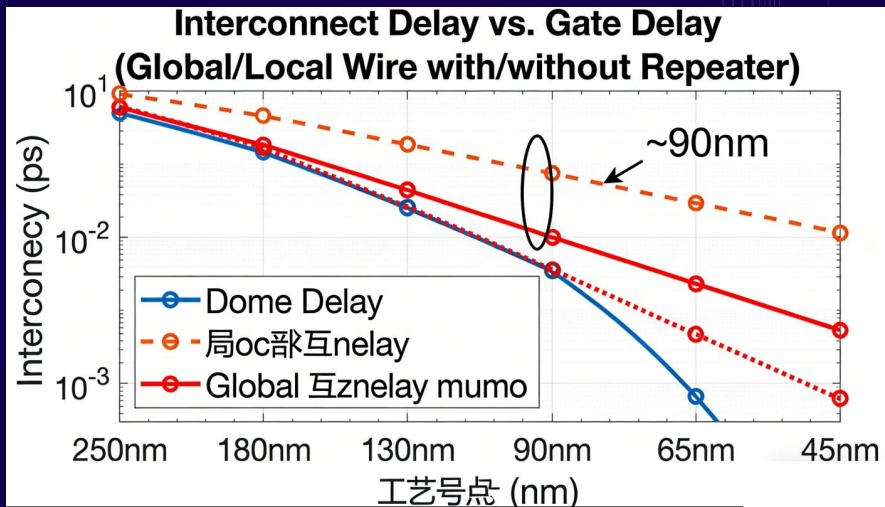


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

Year in which the microchip was first introduced

OurWorldinData.org - Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.



工艺进入 5nm/3nm

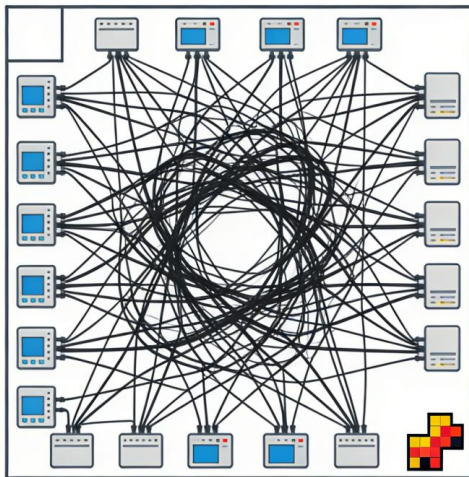
晶体管数量随年份指数上升数量突破100 亿+



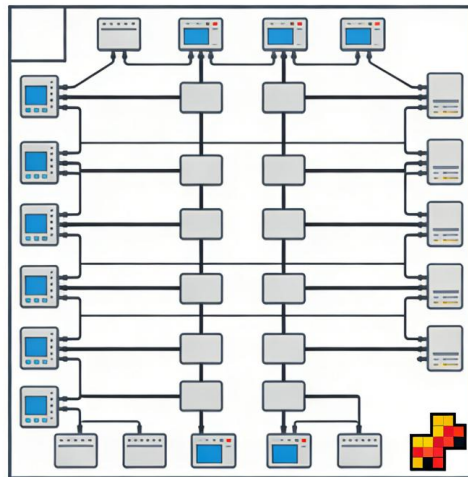
互连线 (Interconnect) 成为性能瓶颈随着工艺节点缩小, 连线延迟占比越来越高, 甚至超过了器件本身



何为布局?



Bad Layout



Good Layout



输入: 网表 (Netlist, 定义了谁和谁连接)、模块尺寸。

输出: 坐标 (x_i, y_i)

目标:

- 无重叠 (No Overlap)
- 线长最短 (Min Wirelength)
- 面积最小 (Min Area)



问题挑战 → 模拟退火算法



物理现象： 固体退火 (Solid Annealing)。

原理： 先加热使原子剧烈热运动（无序），再缓慢降温，原子趋于有序，系统内能降至最低。

数学映射：

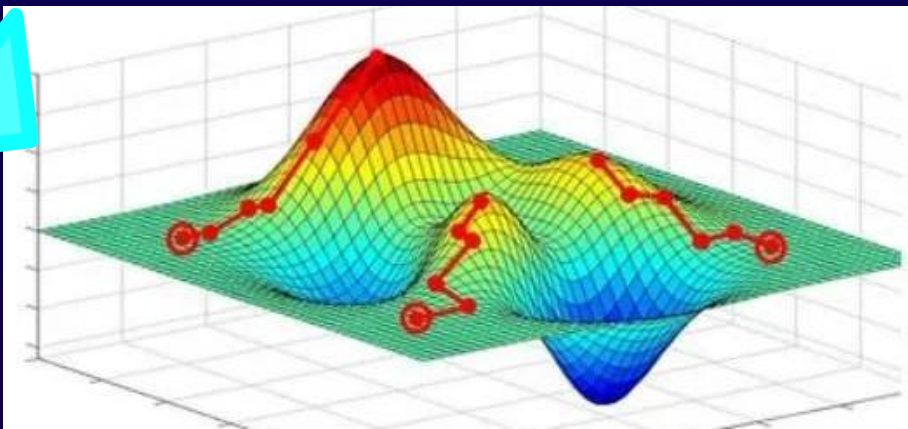
1. 内能-----总线长
2. 温度 T -----控制参数
3. 原子移动-----模块坐标变换

解空间： 假设有

N 个模块，排列组合的数量是指数级的。

暴力穷举： 如果有 20 个模块，穷举需要几千年。

贪心算法的局限： 容易陷入 局部最优。



PART2
数学建模



符号说明与变量定义

1.集合与索引

- $M = \{1, 2, \dots, N\}$: 待布局的矩形宏模块集合, 其中 N 为模块总数。
- $i, j \in M$: 表示模块的索引。

2.模型参数

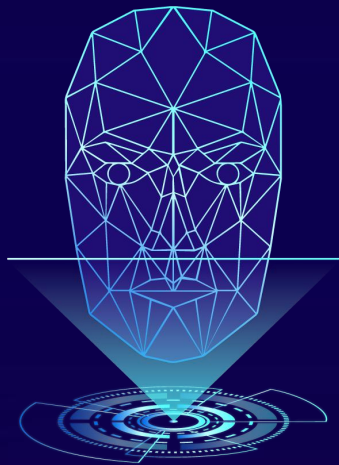
- W_{chip}, H_{chip} : 芯片布局区域 R 的宽度和高度。
- w_i, h_i : 第 i 个模块的固定宽度和高度。
- c_{ij} : 模块 i 与模块 j 之间的连接权重, 若无连接则 $c_{ij} = 0$ 。通常 $c_{ij} = c_{ji}$, 且 $c_{ii} = 0$ 。

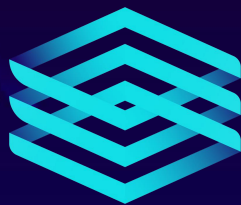
3.决策变量

- x_i : 第 i 个模块左下角的横坐标。
- y_i : 第 i 个模块左下角的纵坐标。

因此, 决策变量集合为:

$$X = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$





目标函数

总目标函数 E 定义为:

$$\text{Min}E = L_{total} + \lambda \cdot P_{overlap}$$

其中:

L_{total} 是加权总连线长度; $P_{overlap}$ 是总重叠面积罚函数; λ 是惩罚系数, 且 $\lambda \gg 0$, 用于控制对重叠的容忍度

1. 加权总连线长度 L_{total} :

采用半周长线长模型 (HPWL) 估算, 即两点间的曼哈顿距离。

因此加权总连线长度 L_{total} 的表达式为:

$$L_{total} = \sum_{i=1}^{N-1} \sum_{j=i+1}^N c_{ij} \cdot (|x_i - x_j| + |y_i - y_j|)$$

2. 重叠惩罚项

两个矩形 i 和 j 在 x 轴方向的重叠长度 Δx_{ij} 为:

$$\Delta x_{ij} = \max(0, \min(x_i + w_i, x_j + w_j) - \max(x_i, x_j))$$

同理, 在 y 轴方向的重叠长度 Δy_{ij} 为:

$$\Delta y_{ij} = \max(0, \min(y_i + h_i, y_j + h_j) - \max(y_i, y_j))$$

因此, 总重叠惩罚项 $P_{overlap}$ 为所有模块对重叠面积之和:

$$P_{overlap} = \sum_{i < j} (\Delta x_{ij} \cdot \Delta y_{ij})$$



模型总结

边界约束:

所有模块必须完全位于芯片区域 R 内部。这意味着模块的左边界不能小于0，右边界不能超过芯片宽度:

$$\begin{cases} 0 \leq x_i \leq W_{chip} - w_i, & \forall i \in \{1, \dots, N\} \\ 0 \leq y_i \leq H_{chip} - h_i, & \forall i \in \{1, \dots, N\} \end{cases}$$

综上所述，该布局问题的数学优化模型如下:

$$\begin{aligned} \min \quad & E(X) = \sum_{i < j} c_{ij} (|x_i - x_j| + |y_i - y_j|) + \lambda \sum_{i < j} O_{ij}(X) \\ \text{s. t.} \quad & 0 \leq x_i \leq W_{chip} - w_i, \quad \forall i \\ & 0 \leq y_i \leq H_{chip} - h_i, \quad \forall i \end{aligned}$$



Metropolis接受准则

1. 数学表达式

假设当前状态为 S_{old} ，能量（目标函数值）为 E_{old} ；产生了一个新状态 S_{new} ，能量为 E_{new} 。

能量差定义为： $\Delta E = E_{new} - E_{old}$

Metropolis准则定义了接受新状态的概率 P ：

$$P(\text{accept}) = \begin{cases} 1, & \text{if } \Delta E < 0 \text{ (更优解, 直接接受)} \\ e^{-\frac{\Delta E}{T}}, & \text{if } \Delta E \geq 0 \text{ (劣解, 按概率接受)} \end{cases}$$

其中：

- $T(\text{Temperature})$ ：当前系统的“温度”，是一个控制参数。
- ΔE ：新旧状态的成本差（在本题中即：线长增量 + 重叠惩罚增量）。



Metropolis接受准则

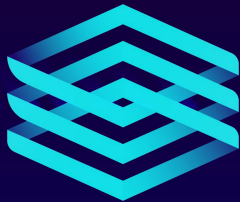
2. 物理意义解析

A. 劣解有多劣:

- 如果 ΔE 很大 (新布局极其糟糕, 导致线长暴增或严重重叠), 那么指数项 $-\frac{\Delta E}{T}$ 会是一个很大的负数, 导致 $P \approx 0$ 。
- 如果 ΔE 很小 (只比当前稍差一点点), P 会比较大。

B. 温度有多高:

- 高温阶段: 当 T 很大时, $\frac{\Delta E}{T} \approx 0$, 于是 $e^0 = 1$ 。
- 低温阶段: 当 T 趋近于0时, 只要 $\Delta E > 0$, $\frac{\Delta E}{T}$ 就会趋向于无穷大, 导致 $e^{-\infty} \approx 0$



Metropolis接受准则

3. 本题执行逻辑:

A. 扰动:

- 随机移动一个模块 m_i , 计算新的总线长和重叠罚分。

B. 判断:

- 如果新布局的总成本 E_{new} 小于旧成本 E_{old} , 无条件更新布局。
- 如果 $E_{new} > E_{old}$ (布局变差了), 生成一个 $[0,1]$ 之间的随机数 r 。
- 如果 $r < e^{-\frac{E_{new}-E_{old}}{T}}$, 则接受这个变差的布局 (冒险跳出局部最优)。
- 否则, 拒绝移动, 保持模块位置不变。



PART3

算法分析与代码实现





模拟退火全过程

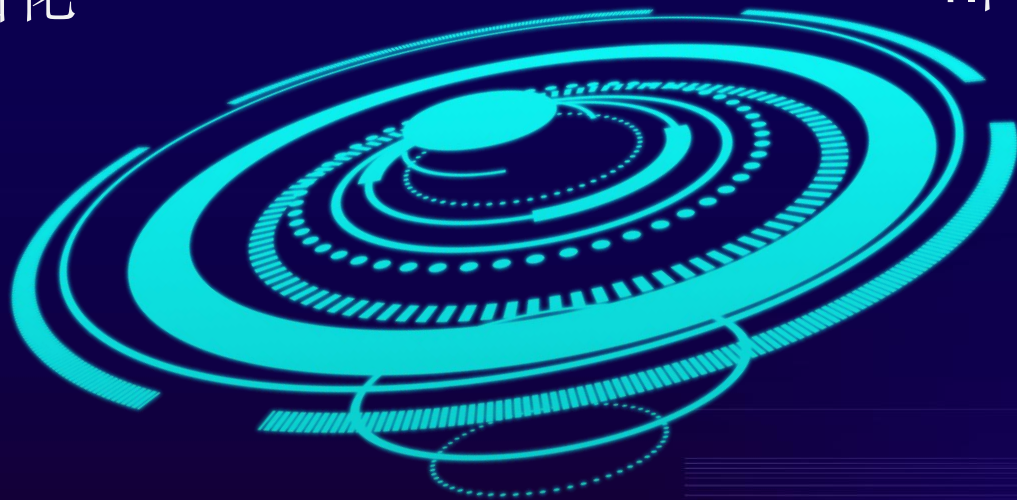
1.初始化

2.生成新解

3.打分

4.降温

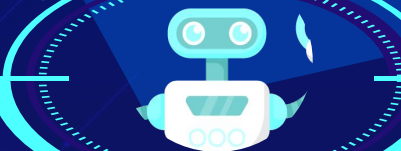
循环,
直至降温至0结束





1.初始化

```
# 步骤1: 计算模块优先级 (总连接权重*面积)
priority_scores = {}
for module in modules:
    total_c = np.sum(connection_matrix[module.id])
    module.total_connection = total_c
    priority_scores[module.id] = total_c * (module.width * module.height)
sorted_module_ids = sorted(priority_scores.keys(), key=lambda x: priority_scores[x], reverse=True)
```



沿用贪心策略，
核心是把权重转化为 $\text{total_connection} \times \text{area}$ 来进行评判，作为第一个摆放较优，但在后续显然会出现局部最优导致的全局较劣。在退火实现部分，方便起见以此为初始局面



2.退火-生成新解

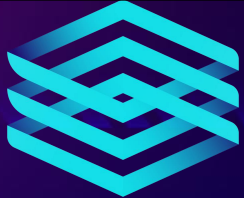
```
# 随机选择一个模块进行扰动
idx = random.randint(0, N_MODULES-1)
module = new_modules[idx]

# 随机调整x、y坐标（步长范围：-STEP_SIZE ~ +STEP_SIZE）
dx = random.randint(-STEP_SIZE, STEP_SIZE)
dy = random.randint(-STEP_SIZE, STEP_SIZE)
new_x = module.x + dx
new_y = module.y + dy

# 约束：确保模块在芯片内
new_x = max(0, min(new_x, CHIP_WIDTH - module.width))
new_y = max(0, min(new_y, CHIP_HEIGHT - module.height))
```



生成扰动后新解，依靠随机概率，在指定步长内产生一个新的合法解（注意此时不出界即可）



3.退火-打分

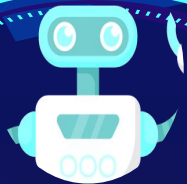
```
# 1. 加权总连线长度 (HPWL模型)
hpwl_cost = 0.0
for i in range(N_MODULES):
    for j in range(i+1, N_MODULES):
        m_i = modules[i]
        m_j = modules[j]
        c_ij = connection_matrix[i][j]
        manhattan_dist = abs(m_i.x - m_j.x) + abs(m_i.y - m_j.y)
        hpwl_cost += c_ij * manhattan_dist

# 2. 重叠惩罚
overlap_penalty = 0.0
for i in range(N_MODULES):
    for j in range(i+1, N_MODULES):
        if check_overlap(modules[i], modules[j]):
            overlap_penalty += PENALTY

total_cost = hpwl_cost + overlap_penalty
```

核心部分:

退火的核心在于对于差结果的容忍，若此时布局代价最优则直接接受，若产生的随机数小于代价P 则也接受此结果。好处在于一定限度内引入了局部较差解，在增加了后续可能性的同时也剪枝了许多差解





4.退火-降温

```
# 步骤4: 降温  
T *= ALPHA  
iter_num += 1  
cost_history.append(current_cost)
```



通过此步骤让后续程序只接受优秀的解，一步步缩小考虑范围，最终找到全局最优解（其中迭代速度 $\alpha=0.96$ 固定值）

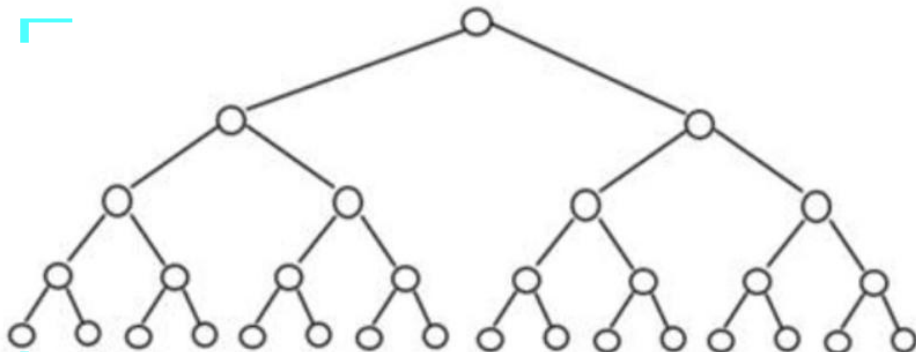


效果对比

在决策树（多叉树）中，**贪心**每次只保留当前子节点中的最优节点，继承为父节点，若不能保证贪心策略全局一致且最优，则容易由局面的扰动导致找不到正确的叶子结点

退火则类似剪枝+bfs，在决策树中会保留较劣解，通过引入T和概率随机数让优化过程中能保留找到最优局面的节点，而T的缩小以及P的限制又对于此决策树剪枝，降低时间复杂度

关键区别：贪心容易因扰动而路径错误，一条路走到黑；退火由多解+剪枝对尽可能多的局面分析，在概率上大大增加了找到解的可能性





PART4

数据可视化与结果分析





初始布局

芯片尺寸

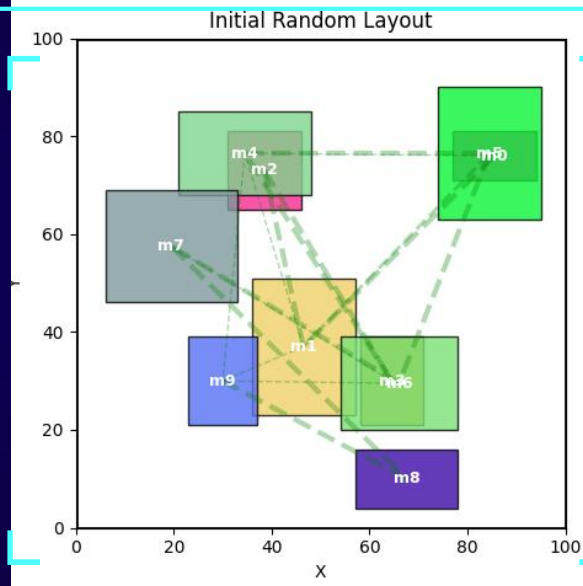
100×100

模块数

N=10

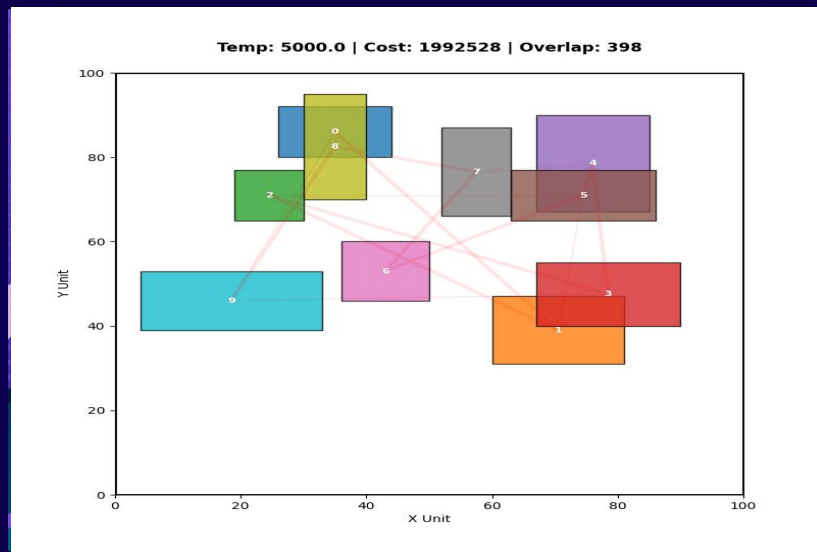
目标

Min (线长+ λ ×重叠)





动态演示



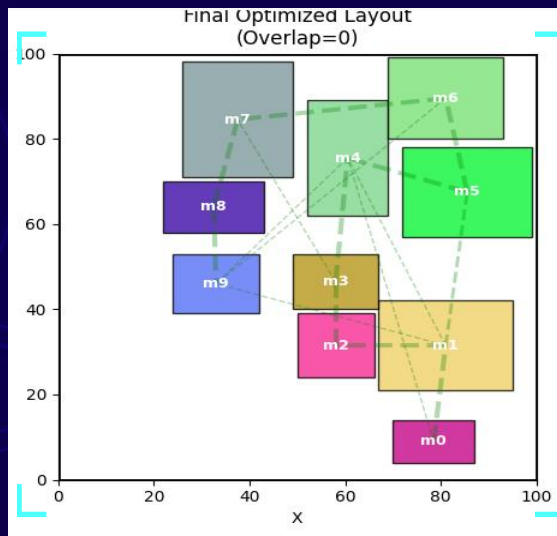
$T=5000 \rightarrow 1$ (降温过程)

红色线 = 存在重叠

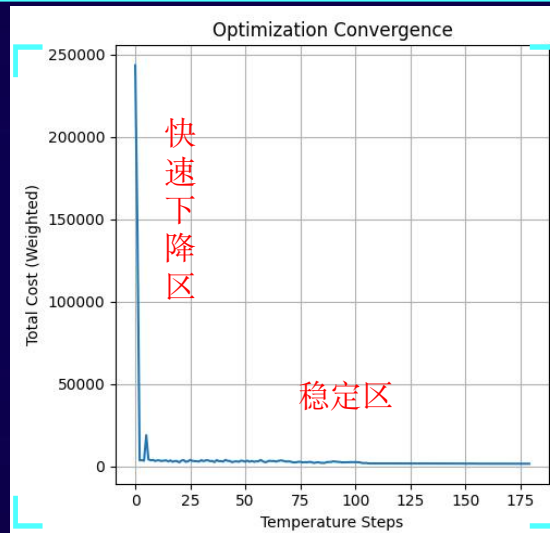
绿色线 = 没有重叠



最终布局



重叠区域为0
且总线长最小





PART5

模型优化与对比



采用局部最优策略,按 “总连接权重 × 面积” 计算模块优先级, 高优先级模块先放置;

```
def calculate_priority(modules, connection_matrix):
    priority_scores = {}
    for module in modules:
        total_c = np.sum(connection_matrix[module.id])
        module.total_connection = total_c
        # 优先级得分 = 总连接权重 * 模块面积
        priority_scores[module.id] = total_c * (module.width * module.height)
    sorted_module_ids = sorted(priority_scores.keys(), key=lambda x: priority_scores[x], reverse=True)
    return sorted_module_ids
```

贪心算法的问题

第一个模块放在芯片中心, 后续模块仅在已放置模块的 “左 / 右 / 上 / 下” 4 个相邻位置中选择无重叠且连线代价最小的位置;
一旦放置完成, 不再调整已放置模块的位置, 属于 “一次性决策”

```
def calculate_total_cost(modules, connection_matrix):
    # 加权总连线长度
    hpwl_cost = 0.0
    for i in range(N_MODULES):
        for j in range(i+1, N_MODULES):
            m_i = modules[i]
            m_j = modules[j]
            c_ij = connection_matrix[i][j]
            manhattan_dist = abs(m_i.x - m_j.x) + abs(m_i.y - m_j.y)
            hpwl_cost += c_ij * manhattan_dist

    # 重叠惩罚
    overlap_penalty = 0.0
    for i in range(N_MODULES):
        for j in range(i+1, N_MODULES):
            if check_overlap(modules[i], modules[j]):
                overlap_penalty += PENALTY

    # 总代价
    total_cost = hpwl_cost + overlap_penalty
    return total_cost, hpwl_cost, overlap_penalty
```

一、克服“步步为营”的局限性

贪心算法按优先级一个接一个放置模块。如果早期放置的模块占用了后续核心模块的最佳位置，则无法回头修改。而 SA 会随机移动已放置的模块,即使某个模块在初始时位置不佳，在温度 T 较高时，它有概率迁移到更合理的区域。

二、接受“暂时退步”以换取“长远进步”

贪心算法只接受 $\text{new_cost} < \text{old_cost}$ ，因此其容易陷入局部最优。而 SA 算法在高温阶段，即使 new_cost 更大，也有较高概率接受，从而更有可能找到全局最优解。

三、更加精细的布局优化

贪心算法只在已放置模块的上下左右四个邻域寻找位置，搜索空间非常有限。而 SA 可以在 STEP_SIZE 范围内进行连续或细粒度的坐标移动。随着温度 T 降低，移动步长可以进一步微调，实现从“粗放布局”到“精细对齐”的过渡。

模拟退火算法优势

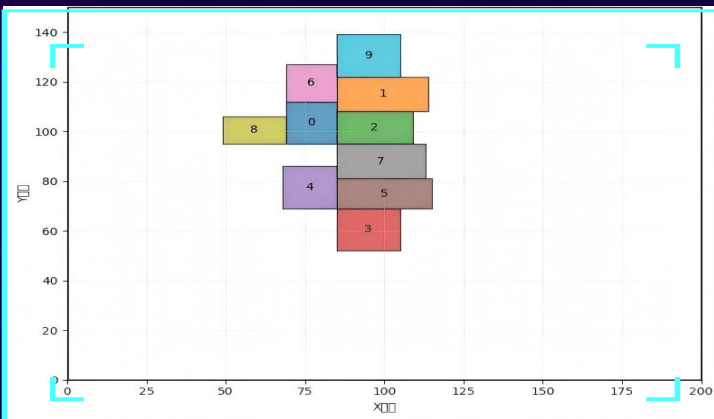


两种算法对比

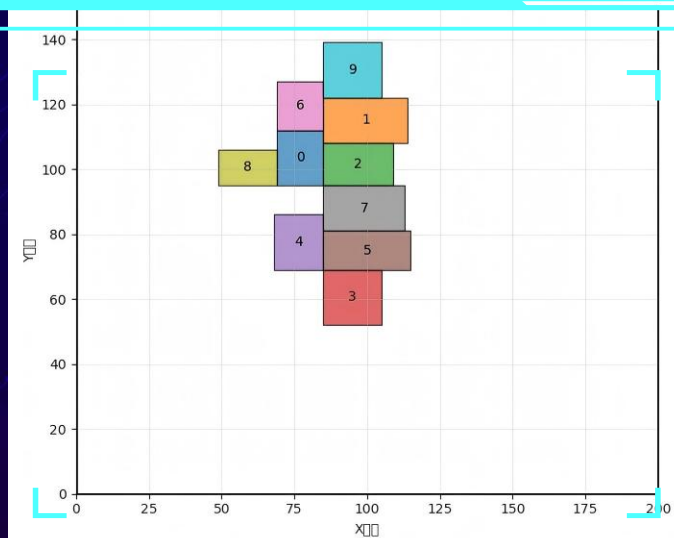
特性	贪心算法	模拟退火算法
搜索策略	局部最优：每一步都选择当前看来最好的位置，一旦放置不再更改。	全局搜索：通过不断“扰动”位置并接受可能变差的解，尝试跳出局部最优。
初始依赖	极度依赖第一个模块的放置位置和优先级排序。	虽然以贪心解为起点，但能通过迭代彻底重构布局。
计算复杂度	低。只需遍历一次模块序列。	高。需要成千上万次迭代
处理约束	强硬约束，若位置重叠则直接舍弃候选点。	软硬结合。通过重叠惩罚将约束转化为代价函数的一部分。



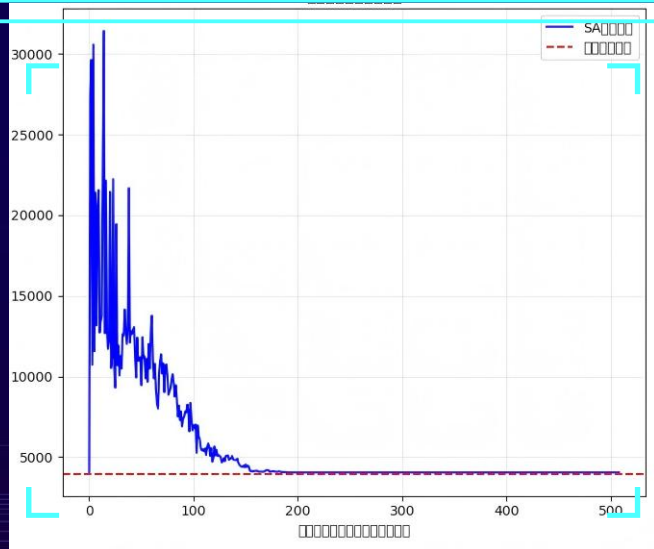
贪心



退火



当模块数量 $N=10$ 时，
可以看到贪心算法和模拟退火算法得到的布局结果相同，因此在模块数量较小时，贪心算法不易陷入局部最优解，无法体现出模拟退火算法的优势。

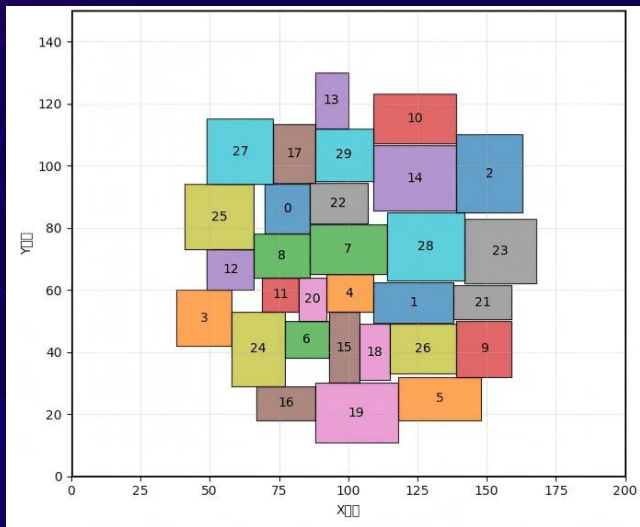


而模拟退火算法得出的模块分布均匀，充分填满芯片区域（无明显大块空闲空间），模块间距离更紧凑——这种排布能直接缩短高连接权重模块间的物理距离，从而降低加权连线长度

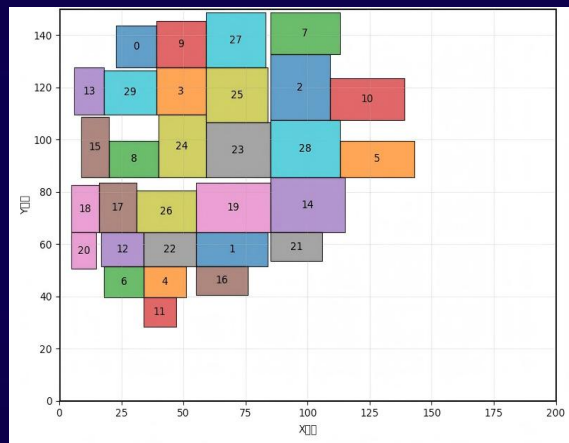


贪心

但当模块数来到 $N=30$ ，
可以明显看到两种算法出现了不同

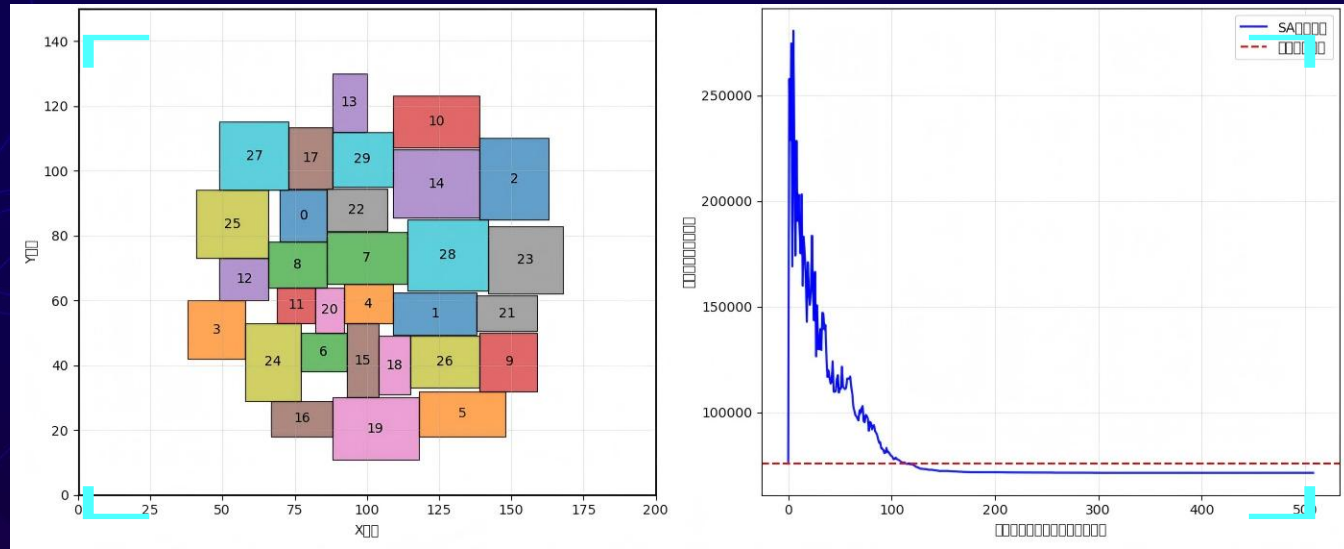


退火



这是贪心算法得到的布局图，可以看到存在明显空闲区域（如 $Y < 40$ 的大片空白），空间利用效率低；同时模块分散排布，增加了高连接权重模块间的距离，直接导致 HPWL 升高。

右侧的代价曲线中，红色的虚线为贪心算法，蓝色的为SA的代价曲线变化。蓝色曲线从初始较高代价快速下降，最终稳定在远低于红色虚线的水平——说明 SA 通过“扰动 - 降温”的迭代优化，突破了贪心算法的局部最优限制，将总代价大幅降低。

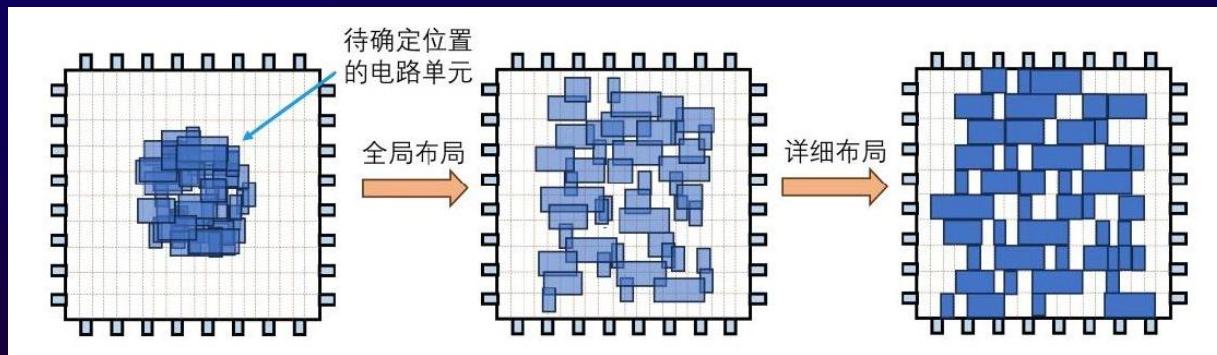


PART6
现实意义



实际应用

算法流派	代表思想	适用场景	优缺点评价
模拟退火 (SA)	物理退火、概率接受	宏单元布局 (Macro Placement)	✓ 全局搜索能力强, 适合复杂约束 ✗ 速度较慢, 不适合亿级规模
解析法	静电场模型、梯度下降	标准单元布局 (Standard Cell)	✓ 速度极快, 工业界主流 ✗ 容易陷入局部最优, 需配合其他算法
强化学习 (RL)	AlphaGo下棋、神经网络	宏单元布局 (Macro Placement)	✓ 无需人类经验, 潜力巨大 ✗ 训练时间长, 黑盒不可解释





现实意义与启示：从算法模型到工业基石



赋能摩尔定律

在百亿晶体管时代，人工布线已不可能。模拟退火等算法是 EDA (电子设计自动化) 软件的核心引擎。

它直接决定了芯片的 PPA (性能、功耗、面积) 指标，是实现 3nm/2nm 先进工艺的关键数学工具。



算法哲学的启示

"退步是为了进步": 模拟退火的核心在于允许暂时接受“劣解”。

启示: 在解决复杂的工程或人生问题时, 为了跳出“局部最优”的舒适区, 往往需要承担短期的风险与倒退, 以换取“全局最优”。



跨学科的数学美学

本模型完美融合了 物理学 (热力学退火)、概率论 (马尔科夫链) 与 工程学 (VLSI设计)。

证明了数学不仅是计算工具, 更是连接物理世界与数字世界的桥梁。

THANKS

组内分工

黄千格: 项目背景&现实意义, ppt制作

丁昊喆: 数学建模

洪康惟: 模型对比与优化

黄佳诚: 算法实现

黄思程: 数据结果可视化

参考文献

[Original Algo] Kirkpatrick, S., et al. "Optimization by Simulated Annealing." Science, 1983.

[Classic Tool] Sechen, C., et al. "The TimberWolf placement and routing package." IEEE JSSC, 1985.

[Textbook] Kahng, A. B., et al. VLSI Physical Design: From Graph Partitioning to Timing Closure. Springer, 2011.

[Modern AI] Mirhoseini, A., et al. "A graph placement methodology for fast chip design." Nature, 2021.